GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	SISTEMAS DISTRIBUIDOS				
TÍTULO DE LA PRÁCTICA:	Programación Distribuida en Java con RMI (Invocación Remota de Métodos)				
NÚMERO DE PRÁCTICA:	**04**	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	2026A
TIPO DE PRÁCTICA:	**INDIVIDUAL**				
	GRUPAL	**X**	**MÁXIMO DE ESTUDIANTES**		**5**
FECHA INICIO:	**04/05/2026**	**FECHA FIN:**	**08/05/2026**	**DURACIÓN:**	**2 horas**
RECURSOS A UTILIZAR: VSCode, Netbeans, Eclipse. *Lenguajes de programación: Java, Python, C++,C#, etc.*					
DOCENTE(s): Mg. Maribel Molina Barriga					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: • Aplicar el funcionamiento de una aplicación RMI para sistemas distribuidos • Implementar una aplicación distribuida que permita interacción cliente-servidor a través de objetos remotos. • Aplicar los conceptos de serialización, stub, skeleton y registro remoto. • Analizar las ventajas del uso de RMI frente a sockets tradicionales.	
TEMAS: • Introducción a la arquitectura RMI • Aplicaciones distribuidas RMI	
COMPETENCIAS	C.e. Identifica de forma reflexiva y responsable, necesidades a ser resueltas usando tecnologías de información y/o desarrollo de software en los ámbitos local, nacional o internacional, utilizando técnicas, herramientas, metodologías, estándares y principios de la ingeniería.

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

RMI

El sistema de Invocación Remota de Métodos (RMI) de Java permite a un objeto que se está ejecutando en una Máquina Virtual Java (JVM - Java Virtual Machine) llamar a métodos de otro objeto que está en otra VM diferente. RMI proporciona comunicación remota entre programas escritos en Java. Si uno de los programas está escrito en

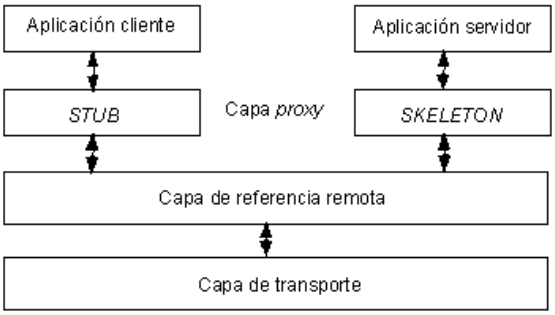
otro lenguaje, se debe de considerar la utilización de IDL (Interface Definition Language - Lenguaje de definición de interfaz) en su lugar.
La tecnología Java IDL para objetos distribuidos facilitan que los objetos interactúen a pesar de estar escritos en lenguaje de programación Java u otro lenguaje tal como C, C++, COBOL, entre otros.

Se basa en la serialización de objetos, el uso de interfaces remotas y el registro de servicios mediante rmiregistry.
Componentes principales

- Interfaz remota: define los métodos que pueden ser invocados remotamente.
- Implementación del servidor: implementa la interfaz y publica el objeto remoto.
- Cliente: busca el objeto remoto y llama sus métodos como si fuera local.
- Registro RMI: mantiene el mapeo entre los nombres lógicos y los objetos remotos.

Ejemplo real: Un sistema bancario distribuido donde las sucursales invocan métodos remotos del servidor central para realizar operaciones como consultas o transferencias.

Arquitectura de JAVA-RMI



La capa 1 es la de **aplicación**, y se corresponde con la implementación real de las aplicaciones cliente y servidor. Aquí tienen lugar las llamadas a alto nivel para acceder y exportar objetos remotos. Cualquier aplicación que quiera que sus métodos estén disponibles para su acceso por clientes remotos debe declarar dichos métodos en una interfaz que extienda java.rmi.Remote. Dicha interfaz se usa básicamente para "marcar" un objeto como remotamente accesible. Una vez que los métodos han sido implementados, el objeto debe ser exportado. Esto puede hacerse de forma implícita si el objeto extiende la clase UnicastRemoteObject (paquete java.rmi.server), o puede hacerse de forma explícita con una llamada al método exportObject() del mismo paquete.

La capa 2 es la capa ***proxy***, o capa stub-skeleton. Esta capa es la que interactúa directamente con la capa de aplicación. Todas las llamadas a objetos remotos y acciones sobre sus parámetros y retorno de objetos tienen lugar en esta capa.

La capa 3 es la de **referencia remota**, es responsable del manejo de la parte semántica de las invocaciones remotas. También es responsable de la gestión de la replicación de objetos y realización de tareas específicas de la implementación con los objetos remotos, como el establecimiento de las persistencias semánticas y estrategias adecuadas para la recuperación de conexiones perdidas. En esta capa se espera una conexión de tipo *stream* (*stream-oriented connection*) desde la capa de transporte.

La capa 4 es la de **transporte**. Es la responsable de realizar las conexiones necesarias y manejo del transporte de los datos de una máquina a otra. El protocolo de transporte subyacente para RMI es JRMP (*Java Remote Method Protocol*), que solamente es "comprendido" por programas Java.

Sintaxis de RMI

RMI reduce la complejidad de la programación distribuida, convirtiendo las tareas de localizar el servidor, realizar la conexión a la red, transferencia de datos, sincronización y propagación de errores en una simple llamada a un método y manejador de excepciones en el cliente. La sintaxis de RMI es la siguiente:

```
try
{    result= remoteInterface.method(args)
}
catch (RemoteException ex)
{
  //manejo de excepciones remotas
}
```

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Es hora de construir un sistema que trabaje en RMI y comprender su funcionamiento. En esta práctica, construiremos un simple servicio de calculadora remota y la utilizaremos con un programa cliente.
Un sistema que trabaja en RMI está compuesto de varias partes:

1. Definición de Interfaz para los servicios remotos.
2. Implementación de los servicios remotos.
3. Los archivos resguardo (***stub).***
4. Host server.
5. Cliente.
6. Instrucciones para compilar y ejecutar el ejemplo.

Pasos para construir el sistema:

1. Escribir y compilar el código java para la interfaz.
2. Escribir y compilar el código java para la implementación de las clases.
3. Compilar los programas Generados.
4. Generar los archivos Stub de la implementación de las clases.
5. Escribir el código java para un servicio de programa host remoto.
6. Desarrollar el código java para un programa cliente RMI.
7. Instalar y correr un sistema RMI.

Definición de la interfaz: El primer paso al crear y compilar el código para el servicio de interfaz. La interfaz "Calculator" define todas las características ofrecidas por el servicio:

Calculator. Java

```
public interface Calculator extends java.rmi.Remote {

public int add(int a, int b) throws java.rmi.RemoteException;
public int sub(int a, int b) throws java.rmi.RemoteException;
public int mul(int a, int b) throws java.rmi.RemoteException;
public int div(int a, int b) throws java.rmi.RemoteException;
}
```

Cliente: El código fuente para el cliente es el siguiente:

CalculatorClient.java

```
import java.rmi.Naming;
import java.rmi.RemoteException;
import java.net.MalformedURLException;
import java.rmi.NotBoundException;


public class CalculatorClient {
public static void main(String[] args) {
int num1 = Integer.parseInt(args[0]);
```

```
int num2 = Integer.parseInt(args[1]);
try {
Calculator c = (Calculator) Naming.lookup("rmi://localhost/CalculatorService");
System.out.println( "The substraction of "+num1 +" and "+num2 +" is: "+ c.sub(num1, num2) );
System.out.println( "The addition of "+num1 +" and "+ num2 +"is: "+c.add(num1, num2) );
System.out.println( "The multiplication of "+num1 +" and "+num2 +" is: "+c.mul(num1, num2) );
System.out.println( "The division of "+num1 +" and "+ num2 +"is: "+c.div(num1, num2) );
}
catch (MalformedURLException murle) {
System.out.println();
System.out.println("MalformedURLException");
System.out.println(murle);
}
catch (RemoteException re) {
System.out.println();
System.out.println("RemoteException");
System.out.println(re);
}
catch (NotBoundException nbe) {
System.out.println();
System.out.println("NotBoundException");
System.out.println(nbe);
}
catch (java.lang.ArithmeticException ae) {
System.out.println();
System.out.println("java.lang.ArithmeticException");
System.out.println(ae);
}
}
}
```

Implementación de la clase: El código para la implementación del servicio remoto está en la clase "CalculatorImpl":

CalculatorImpl.java

```
public class CalculatorImpl extends java.rmi.server.UnicastRemoteObject implements Calculator {
// Implementations must have an explicit constructor
// in order to declare the RemoteException exception
public CalculatorImpl()
throws java.rmi.RemoteException {
super();
}
public int add(int a, int b)


throws java.rmi.RemoteException {
return a + b;
}
public int sub(int a, int b)
throws java.rmi.RemoteException {
return a - b;
}
public int mul(int a, int b)
throws java.rmi.RemoteException {
return a * b;
}
public int div(int a, int b)
throws java.rmi.RemoteException {
return a / b;
}
}
```

Cada objeto contiene las variables de instancia de su clase. Lo que no es tan obvio es que cada objeto también tiene todas las variables de instancia de todas las clases (clase súper padre, abuelo de clase, etc.) Estas variables de clase súper deben inicializar antes de instancias de variables de la clase.

Servidor Anfitrión: Los servicios remotos RMI deben ser hospedados en un servidor de procesos. La clase "CalculatorServer" es un servidor muy simple:

CalculatorServer.java

```
import java.rmi.Naming;
```

```
public class CalculatorServer {
public CalculatorServer() {
try {
Calculator c = new CalculatorImpl();
Naming.rebind("rmi://localhost:1099/CalculatorService", c);
} catch (Exception e) {
System.out.println("Trouble: " + e);
}
}
public static void main(String args[]) {
new CalculatorServer();
}
}
```

Compilar y ejecutar:

Ahora estamos listos para correr el sistema. Nosotros necesitamos tres consolas, una para el servidor, una para el cliente y otra para el RMIRegistry.

Símbolo del sistema - rmiregistry

```
C:\work_eclipse\Calculator-Java-RMI-master>javac *.java
```

Comienza con el comando rmic y la clase de implementación , luego colocar Registry. Debemos estar en el directorio que contiene las clases que escribimos. Aquí introducimos lo siguiente se observa en la figura:

```
C:\work_eclipse\Calculator-Java-RMI-master>rmic CalculatorImpl
Warning: generation and use of skeletons and static stubs for JRMP
is deprecated. Skeletons are unnecessary, and static stubs have
been superseded by dynamically generated stubs. Users are
encouraged to migrate away from using rmic to generate skeletons and static
stubs. See the documentation for java.rmi.server.UnicastRemoteObject.

C:\work_eclipse\Calculator-Java-RMI-master>rmiregistry
```

Luego se ejecuta el Servir:

Símbolo del sistema - java CalculatorServer

```
C:\work_eclipse>cd Calculator-Java-RMI-master

C:\work_eclipse\Calculator-Java-RMI-master>java CalculatorServer
```

Al final se ejecutan los clientes:

Símbolo del sistema

```
C:\work_eclipse\Calculator-Java-RMI-master>java CalculatorClient 7 1
The substraction of 7 and 1 is: 6
The addition of 7 and 1is: 8
The multiplication of 7 and 1 is: 7
The division of 7 and 1is: 7

C:\work_eclipse\Calculator-Java-RMI-master>java CalculatorClient 8 9
The substraction of 8 and 9 is: -1
The addition of 8 and 9is: 17
The multiplication of 8 and 9 is: 72
The division of 8 and 9is: 0
```

III. EJERCICIOS/PROBLEMAS PROPUESTOS

1. Analizar, implementar, ejecutar y probar el código del ejercicio de Ejemplo de RMI en Java, según el **Anexo 1.**

   Para luego realizar lo siguiente:
   - Evaluar resultados obtenidos.
   - Escriba un reporte sobre las tareas realizadas y resultados.
   - Escriba sus conclusiones
2. Utilizando RMI desarrollar una aplicación que permita realizar lo siguiente:
   - Sistema simple de tarjetas de crédito.
   - Definir sus interfaces remotas
   - Crear y compilar las clases de implementación pertenecientes a las clases remotas.
   - Compilar los componentes sustitutos y las clases de los esqueletos mediante la orden rmic.
   - Crear y compilar una aplicación de servidor.
   - Arrancar el RMI Registry y la aplicación del servidor.
   - Crear y compilar un programa cliente para acceder a los objetos remotos.
   - Probar el cliente.
3. Utilizando RMI para un Servicio remoto de conversión de moneda: Crear un servicio remoto que reciba un monto en soles y lo convierta a dólares o euros según una tasa fija. Agregar métodos convertirADolares(double monto) y convertirAEuros(double monto) en la interfaz remota.

IV. CUESTIONARIO

Resolver las siguientes preguntas:

1. ¿Cómo funciona el registro RMI?
2. ¿Cuáles son las subclases para soportar carga dinámica de clases?
3. ¿Qué ventajas y desventajas presenta RMI frente a la comunicación con sockets?
4. ¿Por qué RMI necesita que los objetos sean serializables y cómo se podría escalar este sistema a múltiples servidores RMI?
5. ¿Qué medidas de seguridad se deberían considerar para invocaciones remotas en entornos reales?

V. REFERENCIAS Y BIBLIOGRÁFIA RECOMENDADAS:

[1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.
[2] Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.
[3]Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.
[4] García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: Alfaomega Ra-Ma.
[5] Orfali, R., & Harkey, D. (1998). Client/Server Programming with Java and CORBA. USA: Wiley
[6] RMI: http://www.jtech.ua.es/j2ee/2003-2004/modulos/rmi/sesion01-apuntes.htm
[7] Tutorial oficial de RMI: https://docs.oracle.com/javase/tutorial/rmi/
[8] Oracle Java RMI Documentation: https://docs.oracle.com/javase/8/docs/technotes/guides/rmi/

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN	
TÉCNICAS: *Problemas /Ejercicios propuestos / Preguntas formuladas / Resolución de casos*	**INSTRUMENTOS:** *Lista de cotejo*

CRITERIOS DE EVALUACIÓN

- Identifica el proceso con RMI
- Utiliza las clases aplicativas RMI en Java.

Anexo 1:

MedicineInterface.java

```java
package Medicinas;
import java.rmi.Remote;

public interface MedicineInterface extends Remote {

        public Medicine getMedicine( int amount ) throws Exception;
        public int getStock() throws Exception;
        public String print() throws Exception;
}
```

Medicine.java

```java
package Medicinas;

import java.rmi.server.UnicastRemoteObject;
/**
 * Este es la claes Medicina para este proyecto solo se puede comprar y
 * consultar la lista de medicinas.
 *
 * @author rventurar
 *
 */

public class Medicine extends UnicastRemoteObject implements MedicineInterface {

        private String name;
        private float unitPrice;
        private int stock;

        public Medicine() throws Exception {
                super();
        }

        public Medicine(String name, float price, int stock) throws Exception {
                super();
                this.name = name;
                unitPrice = price;
                this.stock = stock;
        }

        @Override
        public Medicine getMedicine(int amount) throws Exception {
                if (this.stock <= 0)
                        throw new StockException("Stock empty");

                if (this.stock - amount < 0)
                        throw new StockException("Stock not amount of medicine");

                this.stock -= amount;
                Medicine aux = new Medicine(name, unitPrice * amount, stock);
                return aux;
        }

        @Override
        public int getStock() throws Exception {
                return this.stock;
        }
```

```
        public String print() throws Exception{
                return this.name + "\nPrice: " + this.unitPrice + "\nStock: " + this.stock;
        }
}
```

ClienteSide.java

```
package Medicinas;

import java.rmi.Naming;
import java.util.ArrayList;
import java.util.Scanner;
import java.util.HashMap;

public class ClienteSide {
        public static void main(String [] args) throws Exception{
                Scanner sc = new Scanner(System.in);
                StockInterface pharm = (StockInterface) Naming.lookup("PHARMACY");

                System.out.println("Ingresa la opcion\n" +
                                                "1: Listar productos\n" +
                                                "2: Comprar Producto\n");

                int selection = sc.nextInt();

                if( selection == 1 ) {
                        HashMap<String, MedicineInterface> aux =
(HashMap<String,MedicineInterface>)pharm.getStockProducts();
                        for( String key : aux.keySet() ) {
                                MedicineInterface e = (MedicineInterface) aux.get(key);
                                System.out.println(e.print());
                                System.out.println("*--------------*");
                        }
                }

                else if( selection == 2 ) {
                        System.out.println("Ingrese nombre de la medicina");
                        String medicine = sc.next();
                        System.out.println("Ingrese cantidad a comprar");
                        int amount = sc.nextInt();
                        MedicineInterface aux = pharm.buyMedicine(medicine, amount);
                        System.out.println("Usted acaba de comprar");
                        System.out.println(aux.print());
                }
                else {
                        System.out.println("Seleccione una opcion valida");
                }

                sc.close();
        }
}
```

ServerSide.java

```
package Medicinas;
import java.rmi.*;

public class ServerSide {
        public static void main(String [] args) throws Exception {
                Stock pharmacy = new Stock();

                pharmacy.addMedicine("Paracetamol", 3.2f, 10);
                pharmacy.addMedicine("Mejoral", 2.0f, 20);
                pharmacy.addMedicine("Amoxilina", 1.0f, 30);
                pharmacy.addMedicine("Aspirina", 5.0f, 40);

                Naming.rebind("PHARMACY", pharmacy);
                System.out.println("Server ready");
        }
}
```

Stock.java

```java
package Medicinas;

import java.util.HashMap;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class Stock extends UnicastRemoteObject implements StockInterface {
	private HashMap<String,MedicineInterface> medicines = new HashMap<>();

	public Stock() throws RemoteException {
		super();
	}

	public void addMedicine( String name, float price, int stock) throws Exception {
		medicines.put(name, new Medicine(name, price, stock));
	}
	@Override
	public MedicineInterface buyMedicine(String name, int amount) throws Exception {
		MedicineInterface aux = medicines.get(name);
		if (aux == null ) {
			throw new Exception("Imposible to find " + name);
		}

		MedicineInterface element = aux.getMedicine(amount);
		return element;
	}

	@Override
	public HashMap<String, MedicineInterface> getStockProducts() throws Exception {
		return this.medicines;
	}

}
```

StockException.java

```java
package Medicinas;

public class StockException extends Exception{
	public StockException(String msg) {
		super(msg);
	}
}
```

StockInterface.java

```java
package Medicinas;
import java.rmi.Remote;
import java.util.ArrayList;
import java.util.*;

public interface StockInterface extends Remote {
	public HashMap<String, MedicineInterface> getStockProducts() throws Exception;
	public void addMedicine( String name, float price, int stock) throws Exception;
	public MedicineInterface buyMedicine(String name, int amount) throws Exception;
}
```